

I1 security-review follow-up — config-gated HSTS for TLS-terminating-proxy topology

Revision: 1 Last modified: 2026-07-10T00:00:00Z

Finding (I1)

From docs/research/security_headers_adversarial_review_20260710/:
server/internal/api/security_headers.go gated HSTS (and the SPA CSP's upgrade-insecure-requests) SOLELY on
`cfg.TLSCertFile != "" && cfg.TLSKeyFile != ""`. In the common production topology the Go control plane sits BEHIND a TLS-terminating reverse proxy (nginx / cloud load balancer) and receives PLAIN HTTP — so no local cert/key is ever configured — meaning HSTS was silently NEVER sent even though the real client<->proxy connection is genuinely HTTPS.

Fix

Added a config-gated trust flag, `Config.TrustTLSProxy` (env `HELIX_TRUST_TLS_PROXY`, boolean via `strconv.ParseBool`, default false). `Server.tlsEnabled()` now returns true when EITHER the app terminates TLS itself (unchanged cert/key gate) OR the operator has explicitly set `TrustTLSProxy=true`. The flag is a pure operator assertion — no request header (e.g. X-Forwarded-Proto) is ever consulted, so a client cannot spoof HSTS emission over an actually-plaintext hop.

Files changed

- server/internal/config/config.go — added `TrustTLSProxy bool` field + `HELIX_TRUST_TLS_PROXY` env read (via new `getBool` helper, mirroring the existing `getInt64/getDuration` pattern).

- `server/internal/api/security_headers.go` — `tlsEnabled()` gate:
`return (s.cfg.TLSCertFile != "" && s.cfg.TLSKeyFile != "") ||`
`s.cfg.TrustTLSProxy.` Doc comments updated (file-header Tier-A
summary + `tlsEnabled` doc).
- `server/internal/api/security_headers_test.go` — added
`newSecHeadersRouterTrustProxy` helper (parameterises
`TrustTLSProxy`; `newSecHeadersRouter` now delegates to it with `false`)
+ new test `TestSecurityHeaders_HSTS_TrustTLSProxy`.
- `server/internal/config/config_test.go` — added `TrustTLSProxy`
default-`false` assertion to `TestLoadDefaults`, new
`TestLoadTrustTLSProxyOverride`, and a `bad_trust_tls_proxy_bool`
case to `TestLoadInvalidValues`.
- `.env.example` — documented `HELIX_TRUST_TLS_PROXY` (repo root;
there is no `server/.env.example` — env vars are documented at the
repo-root file only, matching the existing `HELIX_MAX_INFLIGHT`
section style. Note: `HELIX_TLS_CERT/HELIX_TLS_KEY` themselves are
not yet documented there either — out of scope for this fix, which
only adds the new var).

Default-OFF byte-identical confirmation

With `HELIX_TRUST_TLS_PROXY` unset, `getBool(..., false)` returns `false`
(same as `getInt64/getDuration` fallback semantics on empty env), so
`cfg.TrustTLSProxy == false` and `tlsEnabled()` reduces to the exact
original expression `s.cfg.TLSCertFile != "" && s.cfg.TLSKeyFile !=`
`""`. Proven by `TestLoadDefaults` (config package) and case (a) of
`TestSecurityHeaders_HSTS_TrustTLSProxy` (api package) below.

§11.4.115 RED → GREEN

RED (fix reverted — `tlsEnabled()` mutated back to the original cert/
key-only expression, `TrustTLSProxy` ignored):

```
=== RUN    TestSecurityHeaders_HSTS_TrustTLSProxy
    security_headers_test.go:289: TrustTLSProxy=true, no local
TLS: HSTS = "", want "max-age=63072000; includeSubDomains"
--- FAIL: TestSecurityHeaders_HSTS_TrustTLSProxy (0.00s)
FAIL
FAIL      github.com/HelixDevelopment/helix_ota/server/internal/
api      0.005s
```

Case (b) — trust flag ON, no local TLS — fails exactly as expected: without the fix, TrustTLSProxy is ignored and HSTS is never emitted. Cases (a) and (c) are unaffected by the mutation (both reduce to the pre-existing cert/key gate), so only (b) fails, precisely isolating the fix's effect.

Immediately after capturing the above, the mutation was reverted (the real fix restored) — confirmed no MUTATED marker remains in the committed file (`grep -c MUTATED security_headers.go → 0`), per §11.4.84/§11.4.115.

GREEN (fix in place — all three cases + full targeted suite):

```
=== RUN    TestSecurityHeaders_TierA_OnEveryResponseClass
--- PASS: TestSecurityHeaders_TierA_OnEveryResponseClass (0.00s)
=== RUN    TestSecurityHeaders_TierA_OnShed429
--- PASS: TestSecurityHeaders_TierA_OnShed429 (0.00s)
=== RUN    TestSecurityHeaders_TierA_OnShed429_RealRouter
--- PASS: TestSecurityHeaders_TierA_OnShed429_RealRouter (0.00s)
=== RUN    TestSecurityHeaders_TierA_OnPanic500_RealRouter
--- PASS: TestSecurityHeaders_TierA_OnPanic500_RealRouter (0.00s)
=== RUN    TestSecurityHeaders_HSTS_TLSGated
--- PASS: TestSecurityHeaders_HSTS_TLSGated (0.00s)
=== RUN    TestSecurityHeaders_HSTS_TrustTLSProxy
--- PASS: TestSecurityHeaders_HSTS_TrustTLSProxy (0.00s)
=== RUN    TestSecurityHeaders_TierB_ScopedToAPI
--- PASS: TestSecurityHeaders_TierB_ScopedToAPI (0.00s)
=== RUN    TestSecurityHeaders_TierB_NotOnSPAAssets
--- PASS: TestSecurityHeaders_TierB_NotOnSPAAssets (0.01s)
=== RUN    TestSecurityHeaders_TierC_SPADocumentCSP
--- PASS: TestSecurityHeaders_TierC_SPADocumentCSP (0.00s)
=== RUN    TestSecurityHeaders_TierC_UpgradeInsecureWhenTLS
--- PASS: TestSecurityHeaders_TierC_UpgradeInsecureWhenTLS (0.00s)
=== RUN    TestSPACSP_CompatibleWithRealBundle
--- PASS: TestSPACSP_CompatibleWithRealBundle (0.00s)
PASS
ok      github.com/HelixDevelopment/helix_ota/server/internal/
api     1.050s
```

TestSecurityHeaders_HSTS_TrustTLSProxy exercises all three cases against the REAL router (`srv.Router()`, via `newSecHeadersRouterTrustProxy`):

- 1. default (TrustTLSProxy unset/false) + no local TLS → HSTS absent (current/original behavior preserved byte-for-byte);

- 1. TrustTLSProxy=true + no local TLS → HSTS present (the fix);
- 1. local TLS configured (TrustTLSProxy=false) → HSTS present (unchanged pre-existing path).

Verification (this cycle, fix in place)

```
$ cd server && GOMAXPROCS=2 nice -n 19 go build ./...
(exit 0)
```

```
$ go vet ./...
(exit 0, no output)
```

```
$ gofmt -l internal/api/ internal/config/
(empty – clean)
```

```
$ GOMAXPROCS=2 nice -n 19 go test -race ./internal/api/ -run
'SecurityHeaders|HSTS|CSP' -count=1 -v
... (11/11 PASS, see GREEN block above)
PASS
```

```
ok      github.com/HelixDevelopment/helix_ota/server/internal/
api     1.050s
```

```
$ GOMAXPROCS=2 nice -n 19 go test -race ./internal/api/ -count=1
ok      github.com/HelixDevelopment/helix_ota/server/internal/
api     13.474s
```

```
$ GOMAXPROCS=2 nice -n 19 go test ./internal/config/ -count=1 -v
=== RUN   TestLoadDefaults
--- PASS: TestLoadDefaults (0.00s)
=== RUN   TestLoadOverrides
--- PASS: TestLoadOverrides (0.00s)
=== RUN   TestLoadTrustTLSProxyOverride
--- PASS: TestLoadTrustTLSProxyOverride (0.00s)
=== RUN   TestLoadInvalidValues
--- PASS: TestLoadInvalidValues/bad_duration (0.00s)
--- PASS: TestLoadInvalidValues/bad_poll_jitter (0.00s)
--- PASS: TestLoadInvalidValues/bad_access_token_ttl (0.00s)
--- PASS: TestLoadInvalidValues/bad_device_token_ttl (0.00s)
--- PASS: TestLoadInvalidValues/bad_max_inflight (0.00s)
--- PASS: TestLoadInvalidValues/bad_int (0.00s)
--- PASS: TestLoadInvalidValues/bad_base64_pubkey (0.00s)
--- PASS: TestLoadInvalidValues/bad_trust_tls_proxy_bool
(0.00s)
```

PASS

ok github.com/HelixDevelopment/helix_ota/server/internal/
config 0.002s

Scope discipline

No git commands were run by this agent (conductor commits per instructions). No files outside the declared scope were touched: `server/internal/config/` `config.go` + `config_test.go`, `server/internal/api/security_headers.go` + `security_headers_test.go`, `.env.example` (repo root), and this EVIDENCE.md. No production-only test hook was added — the new test attaches to the real router via the existing `srv.Router() / NewServer(Options{...})` public seam, following the pre-existing `newSecHeadersRouter` pattern exactly.